

Deeplog

Use LSTM to predict the next token (combine offline and online learning)

Layer Stacking: Like other deep learning models, DeepLog can stack multiple hidden layers to form a "deep LSTM," where the hidden state of one layer becomes the input for the next

Standard Training: It uses standard neural network techniques, such as **one-hot vectors** for input encoding, **categorical cross-entropy loss** for training, and **gradient descent** for optimization

Dual-Model Approach: DeepLog does not use a single LSTM for all tasks. It employs **separate LSTM models** for different data types: one for **log keys** (modeling execution paths as natural language) and others for **parameter value vectors** (modeling performance metrics as multi-variate time series)

Multi-Class vs. Binary Classification: Many traditional anomaly detection models are trained as binary classifiers using both normal and abnormal data. DeepLog is trained as a **multi-class classifier using only normal data**, learning to predict the probability of the "next" log key in a sequence.

Top-g Prediction Logic: Instead of just looking for the most likely next step, DeepLog considers a log entry "normal" if it falls within the **top g most probable candidates** predicted by the model

Parameter Thresholding: For the parameter value model, DeepLog uses a specialized detection method where errors (Mean Square Error) are modeled as a **Gaussian distribution** to determine confidence intervals for anomalies, rather than using a simple fixed threshold

Results

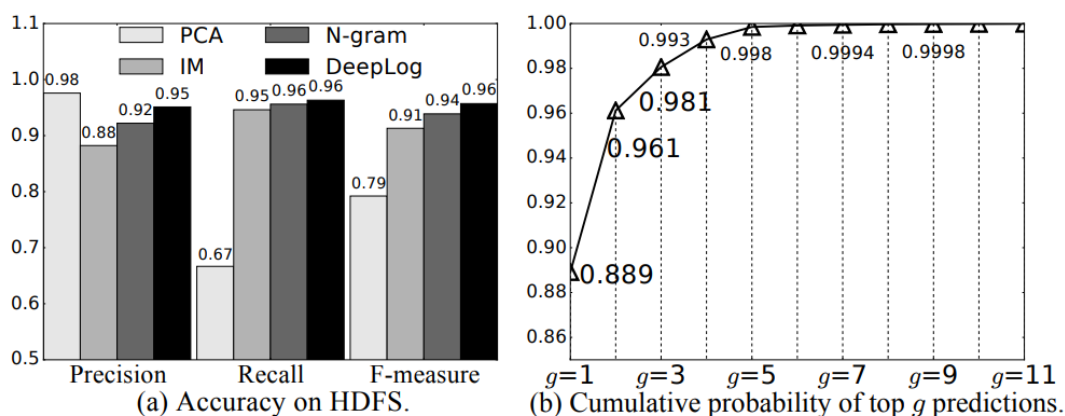


Figure 6: Evaluation on HDFS log.

Difference from ours:

We tested with both one-hot encoding as input and text embedding of log sequences as input.

Single LSTM model for binary classification.

LogAnomaly

Semantic Template Representation (template2Vec): Instead of using numerical indexes for log templates, LogAnomaly uses a novel method called template2Vec.

- It extracts semantic and syntax information from log templates by converting words into vectors using a distributional lexical-contrast embedding (**dLCE**) model,.
- It specifically captures **synonyms and antonyms** (e.g., "up" vs. "down") by querying WordNet and incorporating domain knowledge from operators.
- A template vector is then calculated as the weighted average of these word vectors.

Dual-Pattern Detection with LSTM: LogAnomaly uses **Long Short-Term Memory (LSTM)** networks to learn two distinct patterns simultaneously:

- **Sequential Patterns:** It predicts the next likely log template vector based on a sliding window of previous templates, following the logic that programs follow fixed execution flows,.
- **Quantitative Patterns:** It utilizes **count vectors**—which record the frequency of each template within a window—to learn constant linear relationships (invariants) in program execution, such as the requirement that every "open file" log must eventually be followed by a "close file" log,.

Online Template Approximation: To handle **new log templates** that appear between periodic model retrainings, LogAnomaly uses a matching mechanism, . When a new log is seen, it extracts a "temporary" template and matches it to the **most similar existing template vector**, allowing the system to continue detection without requiring immediate manual feedback or retraining, .

Result

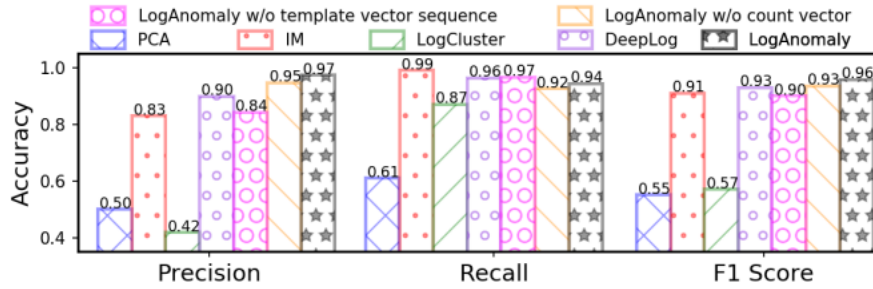


Figure 6: Accuracy on the BGL dataset

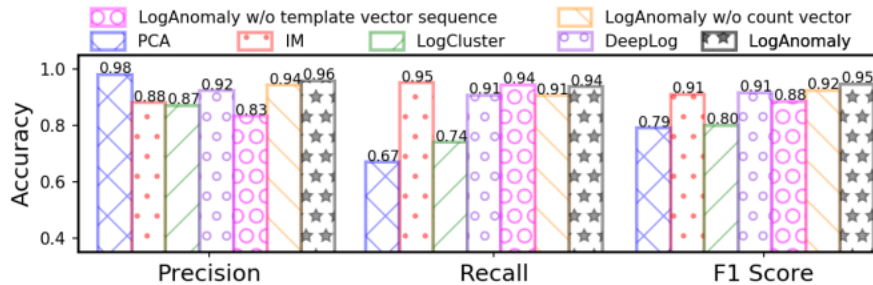


Figure 7: Accuracy on the HDFS dataset

LogGPT

LogGPT is a framework for log anomaly detection that utilizes the **Generative Pre-trained Transformer (GPT)** architecture to model log sequences as natural language. It addresses the limitations of previous models by bridging the gap between training objectives (language modeling) and detection objectives (identifying outliers) through a reinforcement learning strategy.

Log Preprocessing: Raw text-based log messages are first converted into structured **log keys** using a log parser like **Drain**. This transforms the logs into a sequence of vocabulary-like tokens that can be processed by a language model.

GPT-Based Architecture: Unlike previous LSTM-based models (like DeepLog), LogGPT uses a **GPT-2 architecture** with six layers and six heads. This Transformer-based structure allows the model to capture **long-term dependencies** and intricate patterns in long log sequences more effectively than recurrent architectures.

• Two-Stage Training Process:

- **Pre-training:** The model is first trained on normal log sequences to predict the "next" log key given a preceding sequence (prompt). It uses a negative log-likelihood loss to learn the standard patterns of system behavior.

- **Fine-tuning via Reinforcement Learning (RL):** To align the model specifically for anomaly detection, it is fine-tuned using **Proximal Policy Optimization (PPO)**.

- **Novel Top-K Reward Mechanism:** During the RL phase, LogGPT employs a unique reward system. If the actual next log key is within the model's **Top-K predicted candidates**, it receives a **positive reward (+1)**; otherwise, it receives a **negative reward (-1)**. This encourages the model to ensure that normal behavior always falls within its most likely predicted set.

- **Anomaly Detection Logic:** A sequence is flagged as an anomaly if any observed log key in the sequence does not appear in the model's Top-K prediction list.

Method	HDFS		
	Precision	Recall	F-1 score
PCA	0.166 $\pm$ 0.008	0.059 $\pm$ 0.003	0.087 $\pm$ 0.002
iForest	0.043 $\pm$ 0.010	0.422 $\pm$ 0.224	0.078 $\pm$ 0.021
OCSVM	0.058 $\pm$ 0.012	0.910 $\pm$ 0.089	0.108 $\pm$ 0.021
LogCluster	0.996$\pm$0.003	0.368 $\pm$ 0.001	0.538 $\pm$ 0.001
DeepLog	0.793 $\pm$ 0.092	0.863 $\pm$ 0.031	0.824 $\pm$ 0.060
LogAnomaly	0.907 $\pm$ 0.017	0.369 $\pm$ 0.014	0.524 $\pm$ 0.017
OC4Seq	0.922 $\pm$ 0.059	0.758 $\pm$ 0.227	0.808 $\pm$ 0.157
LogBERT	0.754 $\pm$ 0.142	0.749 $\pm$ 0.037	0.745 $\pm$ 0.082
CAT	0.102 $\pm$ 0.022	0.422 $\pm$ 0.082	0.164 $\pm$ 0.034
LogGPT	0.884 $\pm$ 0.030	0.921$\pm$0.066	0.901*$\pm$0.036

The asterisk indicates that LogGPT significantly

TABLE III: Performance of LogGPT with or without reinforcement learning.

Metric	Approach	HDFS	BGL	Thunderbird
Precision	LogGPT w/o RL	0.932 $\pm$ 0.015	0.936 $\pm$ 0.011	0.971 $\pm$ 0.004
	LogGPT	0.884 $\pm$ 0.030	0.940 $\pm$ 0.010	0.973 $\pm$ 0.004
Recall	LogGPT w/o RL	0.790 $\pm$ 0.101	0.975 $\pm$ 0.018	1.000 $\pm$ 0.000
	LogGPT	0.921 $\pm$ 0.066	0.977 $\pm$ 0.018	1.000 $\pm$ 0.000
F-1 score	LogGPT w/o RL	0.853 $\pm$ 0.065	0.955 $\pm$ 0.010	0.985 $\pm$ 0.002
	LogGPT	0.901* $\pm$ 0.036	0.958 $\pm$ 0.011	0.986* $\pm$ 0.002

Significantly outperforms LogGPT w/o RL at the 0.05 level (paired t-test).

LogLLM

LogLLM is a novel framework for **log-based anomaly detection** that simultaneously utilizes both transformer encoder-based and decoder-based Large Language Models (LLMs) to capture the complex semantic information within system logs.

Methodology

The LogLLM framework is designed to streamline log analysis by avoiding traditional log parsers and leveraging the deep semantic understanding of pre-trained models.

- **Preprocessing with Regular Expressions:** Unlike previous methods that rely on log parsers (like Drain or Spell), LogLLM uses **regular expressions** to identify and replace dynamic parameters (e.g., IP addresses, directory paths) with a constant token ("**<*>**"). This approach is more effective for unstable logs where new templates frequently emerge.

- **Hybrid Model Architecture:**

- **BERT (Encoder):** Each preprocessed log message is fed into BERT to extract a **semantic vector**. This effectively summarizes the log message, preventing the memory overflow issues that occur when inputting raw log sequences directly into larger LLMs.

- **Projector:** A linear layer acts as a bridge, mapping BERT's semantic vectors into the **token embedding space** accepted by Llama.

- **Llama (Decoder):** Llama performs the final **log sequence classification**. It receives a prompt containing the log sequence embeddings and a specific query asking if the sequence is "normal or anomalous".

- **Three-Stage Training Procedure:**

- **Stage 1:** Llama is fine-tuned to capture the **answer template** (e.g., responding with "The sequence is normal/anomalous").

- **Stage 2:** The **embedder components** (BERT and the projector) are trained to generate suitable embeddings for Llama to discern anomalies.

- **Stage 3:** The **entire model** is fine-tuned cohesively to ensure accurate performance across all components.

- **Efficiency and Imbalance Handling:** To reduce costs, the framework uses **QLoRA** for 4-bit quantized fine-tuning. It also employs **minority class oversampling** to address data imbalance between normal and anomalous samples.

Result

Methods	Datasets	Log parser	HDFS		
			Prec.	Rec.	F ₁
DeepLog		✓	0.835	0.994	0.908
LogAnomaly		✓	0.886	0.893	0.966
PLELog		✓	0.893	0.979	0.934
FastLogAD		✓	0.721	0.893	0.798
LogBERT		✓	0.989	0.614	0.758
LogRobust		✓	0.961	1.000	0.980
CNN		✓	0.966	1.000	0.982
NeuralLog		✗	0.971	0.988	0.979
RAPID		✗	1.000	0.859	0.924
LogLLM		✗	0.994	1.000	0.997

Data preprocessing (apply to us) (got this part in the Deeplog paper)

We first parse unstructured, free-text log entries into a structured representation, so that we can learn a sequential model over this structured data. As shown by several prior work [9, 22, 39, 42, 45], an effective methodology is to extract a “log key” (also known as “message type”) from each log entry. The log key of a log entry e refers to the string constant k from the print statement in the source code which printed e during the execution of that code. For example, the log key k for log entry e = “Took 10 seconds to build instance.” is k = Took * seconds to build instance., which is the string constant from the print statement `printf(“Took %f seconds to build instance.”, t)`. Note that the parameter(s) are abstracted as asterisk(s) in a log key. These metric values reflect the underlying system state and performance status. Values of certain parameters may serve as identifiers for a particular execution sequence, such as `block_id` in a HDFS log. These identifiers can group log entries together or untangle log entries produced by concurrent processes to separate, single-thread sequential sequences [22, 39, 42, 45]. The state-of-the-art log parsing method is represented by Spell [9], an unsupervised streaming parser that parses incoming log entries in an online fashion based on the idea of LCS (longest common subsequence).

This has been done by the dataset provider.

Toan’s result

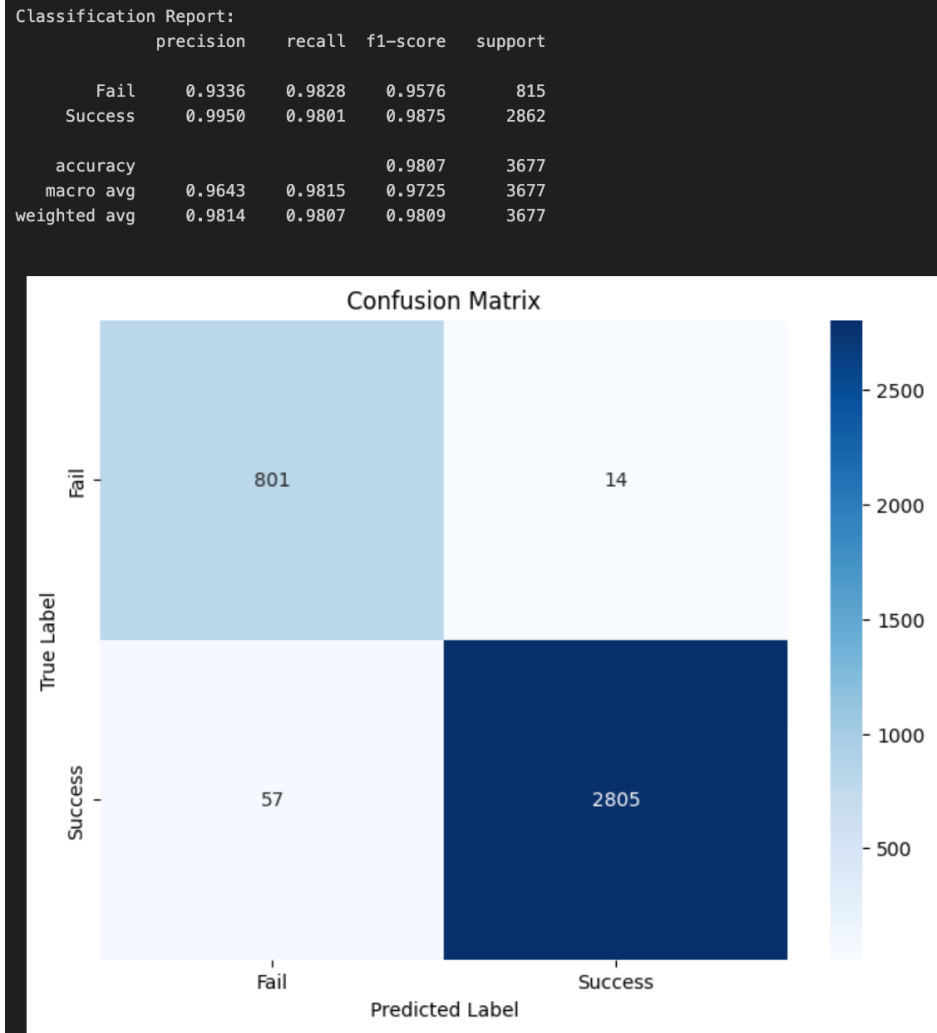
LSTM

Preprocessing

- Load event traces from CSV and remove duplicated sequences per label, then merge Success and Fail.
- Encode labels with sklearn OneHotEncoder into one-hot vectors.
- Left-pad each sequence to MAX_LEN=50 with zeros to create fixed-length sequences; convert to torch tensors.
 - Maximum length of sequence is chosen as 50 based on the distribution of sequence lengths
 - Sequences beyond this length are rare and has dripped effect on buiding even sequences (i.e., padding/truncating) for training. Most short or medium sequences from 1 -> 40 in length are padded up to 200s if this maximum length is not chosen.
- Build torch TensorDataset and split into train/test (80/20); DataLoader with batch_size=64.
- Build torch TensorDataset and split into train/test (80/20); DataLoader with batch_size=64.
- Define an LSTMClassifier that returns the last hidden state via a linear layer to logits.
- Train with Adam (lr=1e-3) and BCEWithLogitsLoss (multi-class one-hot); evaluate with classification_report, confusion matrix, per-class accuracy, overall accuracy.

Embed features with One-hot encoding and build LSTM

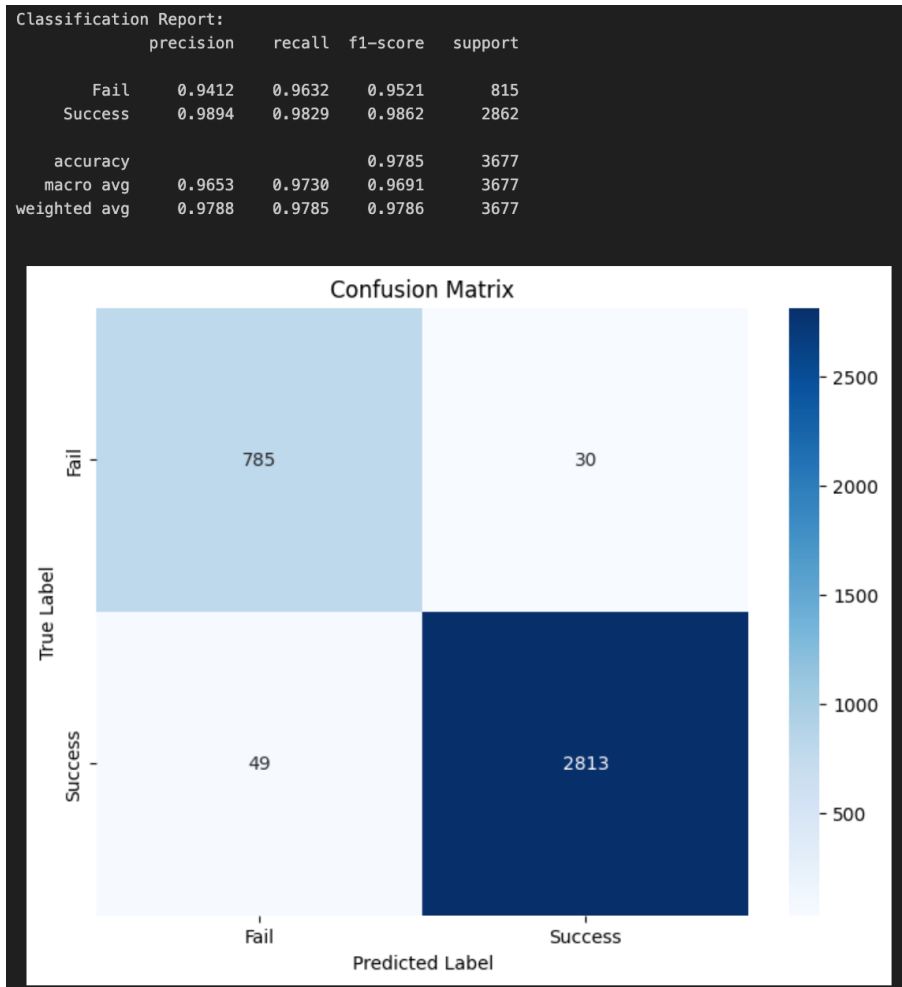
- Feature representation: one-hot vectors for event IDs using sklearn OneHotEncoder.
- ENCODED_DIMEN is inferred from the one-hot encoder; sequences are left-padded to shape (50, ENCODED_DIMEN).
- Model: LSTM input_dim=ENCODED_DIMEN, hidden_dim=16, num_layers=1.
- Training: 6 epochs; tracks average loss per epoch.
- Result:
 - Accuracy for Fail: 0.9828
 - Accuracy for Success: 0.9801
 - Total Accuracy: 0.9806907805276041



Embed features with text embedding using MiniLM and build LSTM

- Feature representation: 384-d sentence embeddings from SentenceTransformer 'all-MiniLM-L6-v2'.
- Loads HDFS.log_templates.csv, encodes each EventTemplate, maps EventId -> embedding, then replaces sequence event IDs with their embeddings.
- Sequences are left-padded to shape (50, 384).
- Model: LSTM input_dim=384, hidden_dim=128, num_layers=1.
- Training: 3 epochs; prints last batch loss per epoch.
- Result:
 - Accuracy for Fail: 0.9632

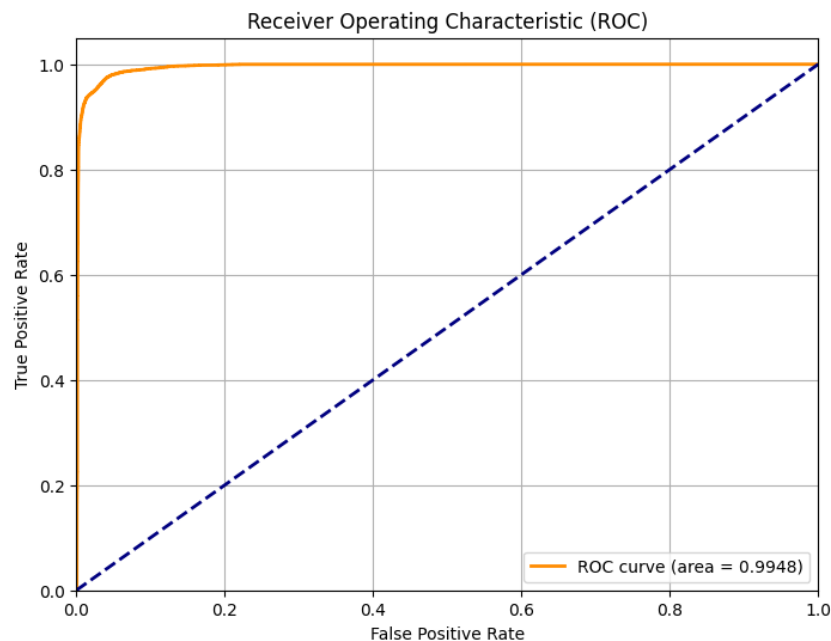
- Accuracy for Success: 0.9829
- Total Accuracy: 0.9785150938264889



Embed features with text embedding using MiniLM and build VAE_LSTM

- Unsupervised anomaly detection using an LSTM-based Variational Autoencoder (VAE).
- Input sequences are left-padded to MAX_LEN=50 for fixed length.
- Labels are one-hot encoded to enable evaluation (not used during training).
- Train the VAE only on “Success” sequences (normal data). Validation and test sets are mixed (Success + Fail) for threshold selection and evaluation.
- Reconstruction error per sample is computed as the MSE summed over sequence and feature dims. A decision threshold is chosen on the validation set using ROC (Youden’s J: $\text{argmax}(\text{TPR}-\text{FPR})$) and then applied to the test set.

- Feature representation: 384-d sentence embeddings from SentenceTransformer 'all-MiniLM-L6-v2'.
 - Build a dict EventId -> embedding from HDFS.log_templates.csv; map each sequence of EventIds to embeddings.
- Data subsampling: drops 40% of “Success” rows to reduce memory usage (as there are many duplications, this action does not affect the overall result).
- Padded tensor shape: (seq_len=50, embed_dim=384).
- Model: LSTM_VAE
 - Encoder: LSTM(input_dim=384, hidden_dim=256, num_layers=2), uses the last layer’s $h_n[-1]$ to produce $\mu/\log\text{var}$ (latent_dim=64).
 - Decoder: linear to hidden_dim, LSTM(hidden_dim→hidden_dim), linear to input_dim; repeats hidden across seq_len to reconstruct.
- Loss: beta-VAE ($\text{MSE} + 0.1 * \text{KLD}$), we use beta-VAE with beta set to 0.1 to prioritize reconstruction rate rather than learning the distribution
- Training: Adam lr=1e-3, EPOCHS=6, train_loader from Success-only.
- Evaluation:
 - Compute reconstruction errors on val, derive optimal threshold via ROC (ROC AUC = 0.9948, Optimal Reconstruction Threshold: 5.6094).



- Result:
 - Accuracy for Fail: 0.9734
 - Accuracy for Success: 0.9591
 - Total Accuracy: 0.9628541772487357

Classification Report:					
	precision	recall	f1-score	support	
Fail	0.8934	0.9734	0.9317	11787	
Success	0.9904	0.9591	0.9745	33494	
accuracy			0.9629	45281	
macro avg	0.9419	0.9663	0.9531	45281	
weighted avg	0.9651	0.9629	0.9634	45281	

